



Universidade do Minho

Licenciatura em Engenharia Informática

Relatório da Fase 1 – Laboratórios de Informática III

Ano Letivo de 2025/2026

Grupo 024

Sara Marques, a111182;

Marta Araújo, a110317;

Teresa Teixeira, a111415.

Índice

1. Introdução	3
2. Trabalho desenvolvido em geral	3
3. Arquitetura do Sistema	3
4. Testes das queries e a sua análise	5
4.1 Contexto	5
4.2 Tabelas	5
4.3 Análise por Query	6
4.3.1 Query 1	6
4.3.2 Query 2	6
4.3.3 Query 3	7
4.4 Conclusão	7
5. Discussão e Análise Crítica	8
6. Conclusão	8

1. Introdução

O presente relatório descreve o trabalho desenvolvido no âmbito da Fase 1 do projeto prático da unidade curricular de Laboratórios de Informática III, desenvolvido pelo grupo 24 constituído por: Sara Marques, Marta Araújo e Teresa Teixeira.

O objetivo central deste projeto consistiu na implementação de um sistema de gestão de informação relativo a aeroportos, voos, aeronaves, passageiros e reservas.

O programa foi desenvolvido na linguagem C e é capaz de carregar dados a partir de ficheiros no formato CSV, submetê-los a um processo de validação sintática e lógica, e armazená-los em estruturas de dados em memória, preparando assim para a subsequente execução de interrogações, as queries, sobre essa informação.

2. Trabalho desenvolvido em geral

Nesta primeira fase, foi dada atenção à implementação de aspetos fundamentais como a modularidade e o encapsulamento. O sistema foi estruturado em vários módulos, separando claramente as responsabilidades de leitura de dados, validação, gestão de entidades e execução de queries. Um dos aspetos mais relevantes do trabalho desenvolvido reside no mecanismo de validação implementado, que assegura a integridade dos dados carregados, rejeitando registos inválidos e registando os respectivos erros em ficheiros.

Adicionalmente, foi estabelecido um módulo de testes automáticos, que permite determinar com rigor o funcionamento correto de todo o sistema.

3. Arquitetura do Sistema

Seguindo os princípios da modularidade, o programa está organizado em componentes bem definidos que interagem entre si.

O módulo principal, “programa-principal” inicia o processo de carregamento dos dados.

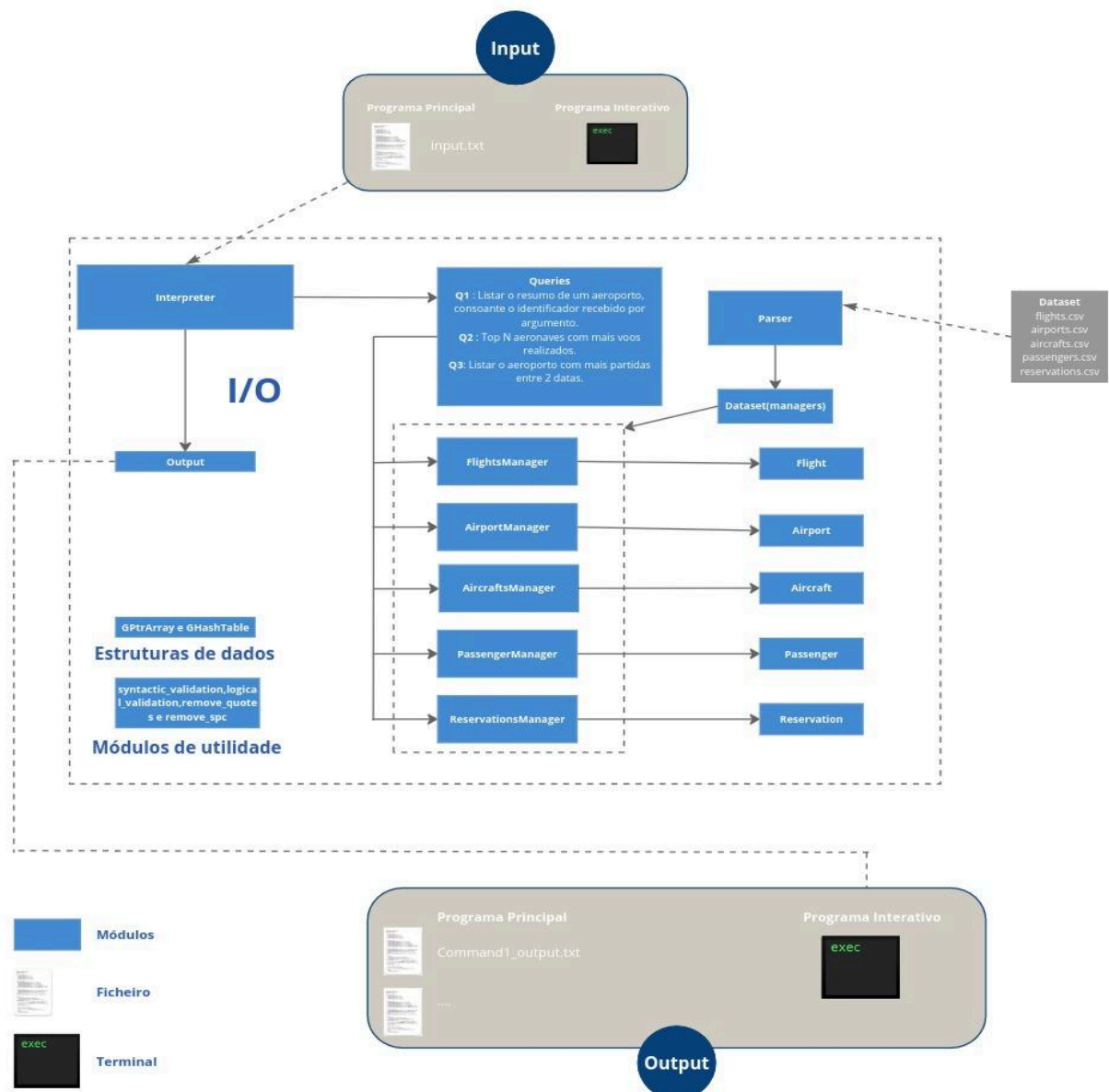
A leitura e o parsing dos ficheiros CSV são da responsabilidade do módulo ‘parser’, que percorre cada linha dos ficheiros de entrada e extrai os dados.

Após a extração, estes dados são submetidos a um processo de validação. A validação sintática, implementada no módulo “syntactic_validation”, verifica a conformidade dos dados com os formatos esperados, como datas no padrão “aaaa-mm-dd”, endereços de email válidos, códigos de aeroporto IATA com três letras maiúsculas, entre outros. Paralelamente, a validação lógica, no módulo “logical_validation”, garante a consistência semântica dos dados, assegurando, por exemplo, que o aeroporto de destino de um voo é diferente do de origem, que uma aeronave referenciada num voo existe de facto no sistema, ou que os tempos de chegada são coerentes com os de partida.

Os registos que passam com sucesso nesta dupla validação são então encapsulados em objetos das respectivas entidades (voos, aeroportos, etc.) e armazenados em estruturas de dados geridas por módulos específicos (“flights_manager”, “airports_manager”, etc.). Estes gestores são responsáveis pelo armazenamento e pela disponibilização dos dados para operações futuras.

A inovação central da nossa arquitetura é a forma como os dados válidos são armazenados. Em vez de listas ou arrays, utilizamos “hashtables” da biblioteca GLib como estrutura de armazenamento principal nos gestores de entidades. Por exemplo, o “FlightsManager” utiliza uma hashtable onde a chave é o “flight_id” e o valor é a estrutura “Flight” correspondente. A mesma abordagem foi aplicada a aeroportos (indexados pelo código IATA), aeronaves (pelo tail number), passageiros (pelo número de documento) e reservas (pelo reservation id). Esta decisão garante que operações de pesquisa, como verificar a existência de uma aeronave para um voo ou de um passageiro para uma reserva, sejam realizadas de forma muito mais eficiente.

A figura abaixo ilustra de forma simplificada a arquitetura implementada:



4. Testes das queries e a sua análise

4.1 Contexto

Esta parte do relatório tem como objetivo avaliar o desempenho do sistema desenvolvido, medindo a eficiência das operações associadas às 3 queries implementadas. Para isso, foi utilizado o executável programa-testes, responsável por validar automaticamente os resultados produzidos e registar tempos de execução.

Os testes foram realizados em diferentes máquinas pertencentes aos elementos do grupo, de forma a observar variações de desempenho devido às diferenças de hardware e condições de execução. Além disso, os testes foram repetidos várias vezes para assegurar estabilidade nos resultados.

Os resultados recolhidos são apresentados sob a forma de tabelas, acompanhados de uma análise crítica que procura relacionar o comportamento observado com as estruturas de dados e algoritmos utilizados na implementação.

4.2 Tabelas

Tabela com as características das diferentes máquinas:

Características:	Computador 1	Computador 2	Computador 3
Modelo Hardware	Lenovo Yoga Pro 7 14IMH9	Macbook Pro	MSI Cyborg 15 A13V
Memória	32.0 GiB	8.0 GiB	16.0GiB
Processador	Intel® Core™ Ultra 7 155H × 22	Apple M3	13th Gen Intel® Core™ i5-13420H x 12
Gráficos	Intel® Arc™ Graphics (MTL)	Apple M3	Intel® Graphics (RPL-P)
Capacidade disco	1.0 TB	494,38 GB	512.1 GB
Sistema operativo	Ubuntu 24.04.2 LTS	MacOS	Ubuntu 24.04.1 LTS
Versão Kernel	Linux 6.14.0-33-generic	25.0.0	Linux 6.14.0-35-generic

Tabela com os tempos das diferentes queries nas diferentes máquinas:

Tempos	Computador 1	Computador 2	Computador 3
Query 1	0.000010 s 0.000008 s 0.000005 s	0.000132s 0.000150s 0.000155s	0.000010 s 0.000008 s 0.000006 s
Query 2	0.000011 s 0.000009 s 0.000006 s	0.000082s 0.000086s 0.000087s	0.000011 s 0.000007 s 0.000010 s
Query 3	0.000009 s	0.000107s	0.000011 s

	0.000008 s 0.000006 s	0.000106s 0.000106s	0.000009 s 0.000010 s
--	--------------------------	------------------------	--------------------------

Tabela com a média dos tempos das diferentes queries nas diferentes máquinas:

Médias dos tempos	Computador 1	Computador 2	Computador 3
Query 1	0.00000766 s	0.000146 s	0.000008 s
Query 2	0.00000866 s	0.000085 s	0.0000093 s
Query 3	0.00000766 s	0.000106 s	0.000010 s

4.3 Análise por Query

4.3.1 Query 1

A Query 1 é a mais simples das três implementadas. O seu objetivo é devolver a informação de um aeroporto, dado o respetivo código IATA. A operação consiste essencialmente numa única pesquisa na estrutura de dados responsável pela gestão dos aeroportos, seguida da escrita dos resultados.

A simplicidade da Query 1 explica os tempos observados: todos os computadores apresentam valores muito próximos entre si e na ordem dos microssegundos. Não existe qualquer etapa de interação sobre estruturas de dados extensas, nem operações complexas como ordenação, filtragem ou contagem. No geral, a Query 1 representa o caso ideal de uma operação simples e direta: pesquisa $O(1)$, validação mínima e escrita imediata.

4.3.2 Query 2

A Query 2 apresenta tempos de execução muito baixos em todas as máquinas testadas, o que reflete a eficiência da abordagem utilizada no código. O objetivo desta query é determinar o Top N de aeronaves com mais voos, ignorando voos cancelados e permitindo opcionalmente filtrar por fabricante. O processo envolve várias etapas: contagem, filtragem, organização e ordenação

No seu conjunto, a Query 2 apresenta tempos muito semelhantes aos das outras queries, mesmo sendo conceptualmente mais complexa do que a Query 1. Isto ocorre porque: a quantidade de informação processada é pequena, o algoritmo evita operações desnecessárias (como contar voos cancelados) e a ordenação, embora passo mais pesado do processo (usando qsort) e apesar desta ser uma operação $O(n \log n)$ é rápida devido ao tamanho reduzido da tabela de aeronaves.

No geral, a Query 2 demonstra uma implementação eficiente e adequada ao problema, mantendo tempos baixos mesmo com múltiplas etapas de processamento. A

estrutura modular, o uso de hashing e o impacto reduzido da ordenação fazem com que o desempenho observado seja coerente com a qualidade do código apresentado.

4.3.3 Query 3

A Query 3 apresenta um nível de complexidade superior às queries anteriores, uma vez que envolve iteração sobre todos os voos do sistema, filtragem por intervalo de datas e contagem de partidas por aeroporto. O seu propósito é identificar qual o aeroporto com maior número de partidas num dado intervalo de tempo, considerando apenas voos não cancelados.

A função inicia-se por percorrer a totalidade da tabela de voos, implementada com uma hash table, o que implica que cada voo armazenado será processado exatamente uma vez. Este comportamento coloca o custo computacional global da query em $O(n)$, onde n corresponde ao número de voos carregados. Esta característica explica o porquê de os tempos observados serem ligeiramente superiores aos das outras queries (especialmente quando comparada com a Query 1).

Os tempos de execução obtidos nas diferentes máquinas refletem precisamente as características desta query. A necessidade de iterar sobre todos os voos e realizar verificações para cada registo contribui para um maior custo computacional, embora ainda baixo graças à eficiência das estruturas hash utilizadas. As diferenças entre máquinas tornam-se ligeiramente mais perceptíveis aqui do que nas outras queries, pois esta operação é mais sensível ao desempenho do processador, especialmente devido à sua capacidade de lidar rapidamente com diversas operações de acesso à memória e manipulação de strings.

No geral, a Query 3 apresenta um desempenho adequado. A sua eficiência deve-se sobretudo ao uso de tabelas hash, à simplicidade das comparações de datas. Os tempos recolhidos confirmam que a implementação é eficiente e capaz de lidar com elevadas quantidades de dados.

4.4 Conclusão

De forma geral, a execução dos testes permitiu avaliar de forma objetiva o desempenho das três queries implementadas e compreender de que forma as escolhas de estrutura de dados e os algoritmos utilizados influenciam o tempo de resposta do sistema. Observou-se que todas as queries apresentam tempos de execução extremamente reduzidos, refletindo a eficiência da utilização de tabelas hash e operações de pesquisa e iteração otimizadas.

5. Discussão e Análise Crítica

A decisão de implementar hashtables desde a primeira fase revelou-se acertada. Ao contrário de uma solução baseada em arrays, que exigiria pesquisas sequenciais ($O(n)$) para validações de existência, a nossa abordagem reduz estas operações a $O(1)$ em casos médios. Isto foi crucial para a validação lógica, onde é frequente a necessidade de verificar se uma chave (ex: 'aircraft_id', 'document_number') existe noutra entidade. O impacto no desempenho do carregamento inicial é notório, especialmente à medida que o volume de dados aumenta.

A integração da GLib trouxe benefícios: para além das hashtables, a GLib fornece funções para manipulação de strings e memória que foram aproveitadas noutras partes do sistema. A gestão de memória, sempre um aspeto crítico em C, foi facilitada pelo uso de funções

como "g_hash_table_new_full", que permite associar funções de destruição às chaves e valores, ajudando a prevenir memory leaks. A ferramenta "valgrind" foi usada extensivamente para validar a correta libertação de todos os recursos.

A nível de implementação, o parser foi um dos módulos mais complexos e demorados.

A função "remove_quotes" e "remove spc" mostrou-se vital para a limpeza dos dados lidos devido à necessidade de lidar com campos entre aspas, listas no formato CSV e remover caracteres extras.

O módulo "programa-testes" foi desenvolvido para comparar automaticamente os ficheiros de output gerados com os resultados esperados. A função "compare_files" realiza uma comparação linha a linha, reportando a primeira diferença encontrada. Adicionalmente, o módulo inclui medições de tempo de execução e, onde suportado, de uso de memória, utilizando "clock_gettime" e "getrusage", respetivamente.

6. Conclusão

Em conclusão, a Fase 1 do projeto foi concluída com sucesso, tendo-se não apenas cumprido os requisitos pedidos pelos docentes. A opção pelas hashtables da GLib desde o início revelou-se uma decisão fundamental, proporcionando ganhos de desempenho significativos e uma gestão de memória mais robusta durante a fase de carregamento e validação dos dados.

As principais lições aprendidas incluem a importância de selecionar as estruturas de dados certas antes da implementação e a eficácia de uma arquitetura modular para gerir a complexidade de um sistema com múltiplas entidades e relações.

Como trabalho futuro, planeia-se aprofundar a análise de desempenho, melhorando os tempos de execução e o aprimoramento do projeto em geral.